

FPGA-DRIVEN INTELLIGENT PARKING MONITORING & CONTROL SYSTEM

Samson Immanuel J,
Assistant Professor
Dept. of ECE,
Sri Krishna College of Technology,
Coimbatore, India

Vidhiya S,
Assistant Professor,
Dept. of CSE,
Sri Krishna College of Technology,
Coimbatore, India

Mohamed Nasthaeen S,
Dept. of ECE,
Sri Krishna College of Technology,
Coimbatore, Tamilnadu, India
727823tuec140@skct.edu.in

Krishna Prasath T,
Dept. of ECE,
Sri Krishna College of Technology,
Coimbatore, Tamilnadu, India
727823tuec124@skct.edu.in

Hamsavardhan S,
Dept. of CSE,
Sri Krishna College of Technology,
Coimbatore, Tamilnadu, India
727823tucs063@skct.edu.in

Joshua Arnold L,
Dept. of CSE,
Sri Krishna College of Technology,
Coimbatore, Tamilnadu, India
727823tucs122@skct.edu.in

Abstract—Vehicle proliferation, infrastructure limitations, and inefficient utilization of available parking spaces increasingly restrict urban parking. The traditional parking systems provide static information and do not track the slot-level occupancy continuously. In this paper we propose an intelligent urban parking system based on FPGA real-time occupancy processing, IoT communication and web-based parking management platform. The occupancy information is processed in parallel and the slot-status information is generated by an FPGA. The vehicle presence at the individual slots is detected by ultrasonic sensors. ESP32 gateway sends the processed information through Wi-Fi to a Spring Boot backend. The software platform monitors the parking inventory, provides real-time availability, suggests appropriate parking, enables booking and navigation and applies demand aware dynamic pricing. A React based interface provides real time parking status and analytics for users and administrators.

The proposed architecture combines embedded hardware and cloud-connected software, and thus, parking information is created from physical occupancy instead of manually or in simulation. The system is designed as a scalable prototype for smartcity parking and can be extended with RFID based access control, automated barriers, occupancy prediction and additional sensing modalities. **Keywords**—Smart parking, FPGA, Internet of Things, ESP32, occupancy detection, dynamic pricing, real-time monitoring, smart city.

I. INTRODUCTION

Rapid urbanization and rising vehicle ownership have brought the parking management as an important component of intelligent transportation system. Drivers may spend considerable time circling in search of a parking spot and available capacity is often unevenly distributed among neighboring facilities. This results in additional traffic, fuel consumption, delays and user frustration. The problem is not just the lack of parking infrastructure but also the lack of timely information about the usage of existing infrastructure.

Smart parking has been investigated in terms of resource allocation, reservations, pricing, sensing and optimization. Geng and Cassandras [1] formulated the parking problem as a dynamic resource-allocation problem in which a parking space is selected based on the distance and price. Kotb et al. pushed this direction further with iParker, which

combines intelligent resource allocation, reservation and pricing in a mathematical optimization formulation [2].

IoT-oriented approaches can provide physical occupancy information, but many systems are still limited to isolated facilities or simple detection and do not combine sensing with allocation, pricing, booking and city-level analytics. Our work closes the integration gap by linking a hardware sensing layer to an existing intelligent parking software architecture.

The key design principle is that occupancy should be sensed physically, processed close to the hardware where appropriate and propagated to the application in real-time. This allows the system to be used to demonstrate the contributions of both ECE and CSE in a single engineering project.

The main contributions of this proposed work include: (1) FPGA based parallel processing of multiple parking-slot sensor inputs; (2) IoT Connectivity through ESP32 to transfer the occupancy information; (3) Real-time backend and web interface for parking availability; (4) Dynamic pricing and recommendation logic based on current occupancy; and (5) Extensible architecture for booking, navigation, analytics, and future predictive models.

II. RELATED WORK

Geng and Cassandras proposed a smart parking system with dynamic resource allocation. Their model distributes and reserves parking resources under a driver objective that considers system-wide utilization while balancing proximity to the destination and parking cost [1]. The approach demonstrates that parking allocation can be treated as an optimization problem rather than a simple nearest-slot search.

Kotb et al. proposed iParker, a smart parking system based on dynamic resource allocation, reservation, and pricing [2]. The work formulates the problem using mixed-integer linear programming with objectives related to driver cost and parking-resource utilization. This provides the conceptual foundation for demand-aware pricing and intelligent allocation used in our system.

Geng and Cassandras also presented a subsequent smart parking system based on resource allocation and reservations, showing how reservations can be incorporated into an urban parking decision process [3]. Other studies have explored IoT-enabled parking, cloudconnected monitoring, and computer-vision-based occupancy detection. Recently, a computer-vision based approach is proposed to

determine parking-space occupancy and parking analytics using image processing and neural-network methods [4].

Despite these advances there is an opportunity of implementation to combine physical slot sensing, FPGA based real-time processing, wireless IoT communication and application level intelligence in one modular system. Therefore, the proposed system is presented as an integrated prototype, not as a replacement for any particular optimization or sensing method.

III. PROBLEM DEFINITION AND OBJECTIVES

The paper tackles the issue of inefficient parking management in urban areas, caused by the lack of visibility of parking occupancy in real time, the fragmentation of parking services and the absence of coordinated allocation and pricing. A practical system would know whether individual spaces are occupied, communicate this state reliably, update users quickly, and use the information to improve parking decisions.

The main goal is to design an intelligent parking platform to connect real parking occupancy to software services. The secondary goals include reducing the effort to find parking, increasing the usage of existing facilities, supporting reservations, providing demand based pricing and enabling administrative analytics.

We evaluate the prototype through measurable engineering metrics that include the accuracy of occupancy detection, the latency from the sensor to the dashboard update, the reliability of communication, the consistency of the booking state, the relevance of the recommendations and the response time. Since the project is in the proposed/prototype stage, numerical performance claims are more targets of evaluation than finished experimental results.

IV. PROPOSED SYSTEM ARCHITECTURE

The architecture consists of the following layers: sensing, FPGA processing, IoT communication, backend services, data management and user applications. Ultrasonic sensors measure the distance at the parking-slot level to detect whether a vehicle is present. The FPGA can process a number of sensor inputs simultaneously, so the slot states can be created in parallel. The processed slot states are transmitted to an ESP32 gateway. The ESP32 provides Wi-Fi connectivity and acts as the interface between the application and the hardware prototype.

We evaluate the prototype through measurable engineering metrics that include the accuracy of occupancy detection, the latency from the sensor to the dashboard update, the reliability of communication, the consistency of the booking state, the relevance of the recommendations and the response time. Since the project is in the proposed/prototype stage, numerical performance claims are more targets of evaluation than finished experimental results.

The architecture consists of the following layers: sensing, FPGA processing, IoT communication, backend services, data management and user applications. Ultrasonic sensors measure the distance at the parking-slot level to detect whether a vehicle is present. The FPGA can process a number of sensor inputs simultaneously, so the slot states can be created in parallel. The processed slot states are transmitted to an ESP32 gateway. The ESP32 provides Wi-Fi connectivity and acts as the interface between the application and the hardware prototype.

V. HARDWARE DESIGN

This separation allows hardware and software modules to be developed and tested independently but still share a common data contract. We propose a miniature prototype of parking with multiple parking slots. Each slot has an ultrasonic sensor which is mounted to sense the

presence of a vehicle. The sensor outputs a distance reading that is compared to a calibrated occupancy threshold by the FPGA.

A stable state is produced only after basic filtering or consecutive-sample validation to eliminate false transitions. The FPGA is used for parallel sensor state processing and not for high level application logic. A hardware state machine can be used to coordinate triggering the sensors, measuring the echoes, comparing against a threshold, and generating the slot-status.

The design is scalable from a small demonstration model to a larger sensor interface since the same processing structure can be scaled across the slots, provided the FPGA I/O and timing constraints are met. The ESP32 reads the slot-status information and converts it into a compact message with the parking-lot identifier, slot identifier, occupancy state, and timestamp or sequence information. The state is then published to the backend via Wi-Fi. Communication failures can be detected by adding a heartbeat mechanism and acknowledgment strategy. Optionally an RFID and barrier subsystem can be added to the entrance. An RFID reader authenticates a parking permit or a registered vehicle; a servo motor is used to represent an automated barrier.

This extension shows how the same backend can coordinate occupancy monitoring and access control. For local visualization, LED indicators may be mounted in individual slots, where one indicator state indicates an available state and another an occupied state. Physical indicators offer real-time feedback during the demonstrations and assist in validating the correlation between the sensor data and the data of the application.

VI. SOFTWARE AND INTELLIGENCE

The backend holds the authoritative state of parking and exposes REST endpoints for users, administrators and hardware gateways. Driver operations are separated from administration by authentication and role-based authorization. Before confirming a reservation, a booking service performs transactional checks to confirm that the slot is available and not already booked by another reservation. Dynamic pricing is occupancy based and not a fixed tariff.

The first model is simple and categorizes occupancy into low, medium and high demand ranges and applies a corresponding price multiplier. This rule-based approach is deliberately transparent and allows for later replacement by a learned pricing model without changing the sensing layer. The recommendation engine assigns a score to available parking options. Distance can be derived from geospatial coordinates. Cost and availability can be derived from the current parking state. You can rank the alternatives on a weighted score to prioritize shorter distance, lower price or higher availability. The application can display parking facilities on an OpenStreetMap-based interface and request routes from OSRM for navigation purposes.

If the selected lot is full upon arrival, the system can also provide alternative facilities. Historical occupancy is stored for analytics, but could be used later for demand forecast. Administrator dashboard contains Occupancy, Usage, Revenue & Demand information. Views at the city level can aggregate parking status over multiple lots to identify areas of high demand and to facilitate integration with future traffic-management systems.

VII. SYSTEM WORKFLOW

- A vehicle is at or approaching a monitored parking space.
- The slot condition is measured by the ultrasonic sensor.

- The FPGA processes the sensor input and decides the occupancy state.
- The ESP32 receives the state and sends it over Wi-Fi.
- The backend authenticates and saves the new state.
- Updates dashboards and connected UIs with WebSocket event.
- The current state is used by the recommendation and pricing engines to rank spaces and determine applicable rates. The user can find a suitable space, book it, make payment using the configured service and get navigation guidance.
- Refreshed on Administrative Dashboards – Occupancy and Usage Analytics

VIII. DYNAMIC PRICING AND RECOMMENDATION MODEL

The initial pricing strategy is interpretable and occupancy-driven. Let $\$O\$$ be the ratio of occupied slots in a parking lot. Example: low occupancy can retain the base tariff, medium occupancy can apply moderate multiplier and high occupancy can apply higher multiplier. The actual thresholds and multipliers are configurable and should be calibrated against observed demand, rather than assumed as universal values.

Each of the available parking options i can be described by distance d_i , normalized cost c_i and availability a_i for recommendation.

These variables can be combined into a weighted score: $S_i = w_d D_i + w_c C_i + w_a A_i$, where the normalized terms are selected such that the lower score is better option. The weights may be adjusted based on user preferences or administrator policy. This approach provides an interpretable baseline before introducing machine learning ranking models.

IX. IMPLEMENTATION AND EVALUATION PLAN

The implementation will be done in phases. First, the FPGA sensor interface will be tested in isolation with occupied and vacant slots in a controlled manner. Secondly, the ESP32 gateway will be tested for reliable transmission under repeated changes in occupancy. Thirdly, we'll integrate the backend and database services and check for consistency between the hardware state and the stored state. At the end the whole workflow will be evaluated using the web interface.

The primary evaluation metrics are: occupancy accuracy, which is the number of correctly identified slot states over the total number of test observations; end-to-end latency, which is the time from a physical occupancy transition to the corresponding dashboard update; packet or message loss rate; booking consistency under concurrent requests; and recommendation response time.

A confusion matrix can be used to differentiate between true occupied, true vacant, false occupied and false vacant decisions for occupancy detection. Latency must be measured in repeated trials under both stable and quickly changing conditions. The final assessment will present measured values of the implemented prototype, without claiming performance a priori. The software-only prototype already describes the application workflow: search, recommendation, pricing, booking, navigation and administrative analytics for parking. The hardware integration we plan to do will replace simulated occupancy with sensor derived occupancy, creating a direct feedback loop from the physical to the digital world.

X. EXPECTED RESULTS AND DISCUSSION

The expected outcome is a working prototype where changes in individual parking slots are reflected in the web application with low

delay. The FPGA layer is intended to deterministically process multiple sensor inputs in parallel, while the ESP32 will be a low-cost wireless gateway. The integrated platform is expected to provide more reliable information on parking availability, than systems that are updated manually. Dynamic pricing can help to spread demand when some facilities become highly occupied.

Recommendations can reduce unnecessary search by considering multiple factors, not just distance. This architecture also lays a foundation for larger deployments. The same backend can receive reports from multiple ESP32 gateways and parking lots can be represented as geospatial entities.

Thus the software is able to evolve from a laboratory scale prototype to a multi-lot system without any change of the fundamental data flow.

Simultaneously, the prototype is limited in practice. Mounting geometry and environmental conditions can affect ultrasonic sensors; Wi-Fi connectivity can suffer from variable latency; and a small prototype cannot capture the complexity of a city-scale parking environment. Calibration, filtering, communication monitoring and controlled experimental evaluation will be used to overcome these limitations.

XI. FUTURE ENHANCEMENTS

- Machine-learning models for short-term parking-demand prediction.
- Camera based occupancy verification or sensor fusion for stronger detection.
- Automatic number plate recognition for easy entry and checking of permits.
- RFID-based access control and automatic barrier operation
- Edge analytics and low power wireless networking for larger deployments.
- Integration with traffic data to suggest parking based on travel time and congestion.
- Multi-city roll out with centralised analytics and policy controls.

XII. DISCUSSION

The scalability is analyzed by adding more slots to be monitored in the prototype or the simulation and observing the usage of the FPGA resources, the amount of messages on the gateway, the time spent by the backend and the behavior of the dashboard when adding more slots. The architecture is assessed and its ease of extension to a larger demonstration model is demonstrated with these measurements.

If the end-to-end latency is required, a timestamp is recorded when the occupancy state changes and another timestamp is recorded when the application state is rendered or received by the client. Average and worst case behavior is obtained from multiple trials. Communication tests can add updates and temporary network interruptions to test recovery.

The evaluation is based on repeated controlled trials. Each slot is tested in known occupancy and vacancy and hardware decision is compared to ground truth for occupancy accuracy. The false-occupied and false-vacant rates are included in addition to accuracy as they have different operational consequences.

XIII. EXPERIMENTAL EVALUATION PLAN

In parallel, the software team develops the parking map, slot-management interfaces, booking workflow, recommendation logic and pricing module. Then, integration testing ensures the entire chain from a physical vehicle or test object to the slot status on display. Final testing measures accuracy, latency, reliability and response of the application.

The project implementation is divided into hardware, communication, backend, frontend and integration. The hardware team first verifies one sensor and FPGA channel and then scales the design to multiple slots. Next, we integrate the communication layer, then the backend data model and the real time update mechanism.

XIV. IMPLEMENTATION ROADMAP

Reliability is handled by validation, timestamps, heartbeats, retries, and consistent state transitions. The system must be able to differentiate between an occupied slot, a free slot and an unknown/stale slot when communication has failed. This prevents a disconnected sensor to be misinterpreted as a reliable vacant space.

You need security at the application boundary and at the device boundary. Driver and administrator functions are protected by user authentication and role-based authorisation. The credentials of registered devices should identify hardware gateways so that arbitrary clients cannot submit false data on occupancy. All sensor and API messages require input validation.

XV. SECURITY AND RELIABILITY

In the final prototype, machine learning should be viewed as a complement, not a prerequisite for basic operation. In the absence of sufficient historical data, the system can still function based on real-time occupancy and rule-based pricing. This staged design mitigates the risk of implementation and provides a clear path to intelligent prediction.

The platform collects historical occupancy events that can be used to support future demand prediction. Features might include the hour, day of week, historical occupancy, parking facility, and location characteristics nearby. A prediction model can be used to predict future occupancy for a chosen time window and support proactive recommendations or pricing.

XVI. MACHINE LEARNING EXTENSION

The recommendation process is run on the current validated occupancy data. The system can also refresh the list of candidates and suggest alternatives if a chosen space is not available. Such behaviour connects real-time sensing to actionable decision support, rather than considering recommendations as a static map-search feature.

The recommendation module ranks the available parking facilities or spots currently based on factors relevant to the driver. Distance, travel time, price and availability can be normalised and rolled into a weighted score. User preferences can alter the weights such that one driver weighs low cost more heavily and the other shorter travel distance.

XVII. INTELLIGENT RECOMMENDATION

The system should also avoid pricing instability. You shouldn't get price changes multiple times just because a sensor is fluctuating for a little bit. Pricing can then be re-computed at controlled intervals or when occupancy has been at a demand band for a predetermined length of time.

This approach is purposely transparent for a prototype. This allows the project team to show that a decision at the application level is influenced by real sensor data. The rule-based model can later incorporate historical demand, time of day, day of week, event schedules or a learned prediction model.

The pricing component turns demand information into a configurable tariff. A baseline implementation can have occupancy bands, such as low, medium and high demand. When the occupancy goes above a configured threshold, the pricing multiplier increases. When the occupancy goes back down, the pricing multiplier decreases toward the base tariff.

XVIII. DYNAMIC PRICING MODEL

The user interface is built around the real time state. When a sensor changes from vacant to occupied, the application should update the corresponding slot without having to refresh the page. This feedback loop from hardware to interface is a major demonstration feature of the integrated project.

The administrator dashboard provides aggregate and facility level views. Administrators can see the occupancy, view the booking activity, analyse the demand patterns and set the parking facilities and pricing policies. It also provides a method for testing whether the physical prototype state is consistent with the software state.

The react application has two main views driver view and admin view. The driver can search for parking, check the availability, weigh the distance and the price, pick a slot, reserve a parking space and get the navigation guidance. The map view delivers a geographic representation of the parking facilities, but does not force the user to make individual searches for each parking facility. Consistency is very important when making a booking.

XIX. BACKEND AND DATA MANAGEMENT

The accuracy of the sensors is not considered in evaluating the reliability of the communication. If the sensor's decision is incorrect, then it would be useless if the update doesn't get to the application. Thus, the gateway needs to re-try any transmission that fails and the back-end should send an acknowledgement for any message received. If you deploy this into production, you'd need to enable encrypted transport and device authentication.

A typical occupancy message includes a parking-lot identifier, slot identifier, occupancy state, device identifier, sequence number and timestamp. The sequence number assists the backend in determining if a message is a duplicate or outdated. It is possible to send a heartbeat message periodically, which can help the server differentiate a gateway that isn't available from a parking lot that doesn't have any changes to its occupancy.

The FPGA prototype and the software platform are connected via the ESP32. It gets the slot-status vector, translates each bit to a parking-slot identifier and sends the records out via Wi-Fi. If a larger deployment is considered, then the gateway can employ a lightweight publish-subscribe mechanism, or it can use HTTP/REST for simpler integration.

XX. USER INTERFACE AND ADMIN DASHBOARD

The backend will need to check whether the requested slot is available and check to see if there is a conflict on the requested slot, before confirming the reservation. The same slot can be successfully reserved by two users in a near-simultaneous reservation request by using database transactions, or something equivalent.

Current state information can be cached in Redis for frequently requested information. This is especially helpful for situations where a parking facility has many users accessing it at the same time. The database is the long-lived source for transactional data, such as bookings, and the cached state is updated as validated occupancy updates arrive to the database.

Entities can be added to the MySQL schema representing users and parking facilities, slots, bookings, payments, occupancy events, and pricing rules. A slot is a part of a parking facility and keeps the most recent status of slots in that parking facility. Occupancy events may be stored as a historical series for analysis and predictive modelling into the future.

The backend is integrated using the Spring Boot layer, which is the central layer. The user operation, parking management, booking and hardware updates are implemented via REST API. WebSockets offer a push channel, enabling clients that are connected to the server to

receive updates about changes in occupancy without making repeated requests to the server.

XXI. IoT COMMUNICATION LAYER

The backend is integrated using the Spring Boot layer, which is the central layer. The user operation, parking management, booking and hardware updates are implemented via REST API. WebSockets offer a push channel, enabling clients that are connected to the server to receive updates about changes in occupancy without making repeated requests to the server.

The accuracy of the sensors is not considered in evaluating the reliability of the communication. If the sensor's decision is incorrect, then it would be useless if the update doesn't get to the application. Thus, the gateway needs to re-try any transmission that fails and the back-end should send an acknowledgement for any message received. If you deploy this into production, you'd need to enable encrypted transport and device authentication.

A typical occupancy message includes a parking-lot identifier, slot identifier, occupancy state, device identifier, sequence number and timestamp. The sequence number assists the backend in determining if a message is a duplicate or outdated. It is possible to send a heartbeat message periodically, which can help the server differentiate a gateway that isn't available from a parking lot that doesn't have any changes to its occupancy.

The FPGA prototype and the software platform are connected via the ESP32. It gets the slot-status vector, translates each bit to a parking-slot identifier and sends the records out via Wi-Fi. If a larger deployment is considered, then the gateway can employ a lightweight publish-subscribe mechanism, or it can use HTTP/REST for simpler integration.

XXII. FPGA SENSOR PROCESSING

The prototype should also be able to withstand occasional and short sensor readings and network outages. The validation and filtering of successive samples (consecutive-sample validation) is therefore desirable for occupancy decisions. Any message sent should be sufficient to determine the parking facility and slot number and the backend should reject the message if it is not formatted correctly or is too old.

Some of the non-functional requirements are low latency for updates, reliable communication, modularity, scalability, maintainability and secure access to application services. The architecture is designed to isolate the sensing logic and application logic so that if the sensor technology changes, the web application doesn't need to be completely redesigned. Likewise, the pricing or recommendation algorithms can be updated without any changes to the FPGA sensor interface.

The system proposed is based on functional and non-functional requirements. Sensing parking occupancy, sending the status of the slot, maintaining the inventory of parking spaces, displaying the status of parking spaces in real time, accepting reservations, calculating the price of the parking space, and allowing administrative monitoring are all functional requirements. The hardware gateway should be able to detect individual slots so that the transition of an occupancy does not cause an incorrect occupancy transition at another space.

XXIII. CONCLUSION

This paper presents an integrated intelligent urban parking architecture that combines FPGA-based occupancy processing, IoT connectivity, and a software platform for parking management. The design moves beyond isolated parking detection by connecting physical slot status with real-time availability, booking, recommendations, dynamic pricing, navigation, and analytics. The division of responsibilities between FPGA hardware, ESP32 communication, backend services, and the web application provides a practical basis for a four-member

ECE and CSE engineering team. The proposed prototype will be validated using occupancy accuracy, latency, reliability, and application-level performance metrics. With future extensions in prediction, computer vision, access control, and smart-city integration, the system can evolve into a broader intelligent parking and mobility platform.

The proposed implementation provides a clear pathway from a small parking prototype to a scalable smart-city architecture. FPGA processing demonstrates real-time embedded intelligence, while ESP32 connectivity and web services connect the physical environment to users. Future work can extend the system with predictive analytics, computer vision, automated access control, and broader smart-city data integration.

The proposed Intelligent Urban Parking System combines FPGA-based occupancy detection, IoT communication, and an intelligent software platform to address the practical limitations of conventional parking management. The architecture supports real-time slot monitoring, dynamic pricing, recommendations, reservations, navigation, and administrative analytics.

XXIV. SYSTEM REQUIREMENTS AND DESIGN CONSIDERATIONS

REFERENCES

- [1] Y. Geng and C. G. Cassandras, "Dynamic Resource Allocation in Urban Settings: A 'Smart Parking' Approach," *Proc. IEEE International Symposium on Computer-Aided Control System Design (CACSD)*, pp. 1–6, 2011. DOI: 10.1109/CACSD.2011.6044566.
- [2] A. O. Kotb, Y.-C. Shen, X. Zhu, and Y. Huang, "iParker—A New Smart Car-Parking System Based on Dynamic Resource Allocation and Pricing," *IEEE Trans. Intell. Transp. Syst.*, vol. 17, no. 9, pp. 2637–2647, 2016. DOI: 10.1109/TITS.2016.2531636.
- [3] Y. Geng and C. G. Cassandras, "New 'Smart Parking' System Based on Resource Allocation and Reservations," *IEEE Trans. Intell. Transp. Syst.*, vol. 14, no. 3, pp. 1129–1139, 2013.
- [4] S. G. et al., "Smart Parking System with Computer Vision," *International Research Journal on Advanced Engineering Hub*, vol. 2, no. 12, pp. 2816–2820, 2024. DOI: 10.47392/IRJAEH.2024.0389.
- [5] M. Idris, Y. Leng, E. Tamil, N. Noor, and Z. Razak, "Car Park System: A Review of Smart Parking System and Its Technology," *Information Technology Journal*, 2009.
- [6] T. N. Pham, M.-F. Tsai, D. B. Nguyen, C.-R. Dow, and D.-J. Deng, "A Cloud-Based Smart-Parking System Based on Internet-of-Things Technologies," *IEEE Access*, vol. 3, pp. 1581–1591, 2015.